

EXPERIMENT 17

PYTHON FLASK API – DOCKER CONTAINERIZATION AND KUBERNETES DEPLOYMENT

Aim: To create a simple Python Flask API, containerize it using Docker, build and push the Docker image to Docker Hub, and deploy the application on Kubernetes using Deployment and Service manifests with NodePort access.

Software Requirements

- Python 3 and Flask
- Docker
- Docker Hub account
- Kubernetes / kind cluster
- kubectl command-line tool
- Web browser or API client

Procedure

- 1. Create the Flask API:** Write an **app.py** file with basic API endpoints such as a home endpoint and a health/status endpoint.
- 2. Create requirements.txt:** Add Flask as the application dependency.
- 3. Create the Dockerfile:** Use a Python base image, install dependencies, copy the application files, expose the Flask port, and start the Flask application.
- 4. Build the Docker Image:** Build the application image using the Docker build command.
- 5. Push to Docker Hub:** Tag the image with the Docker Hub repository name and push it to Docker Hub.
- 6. Create Kubernetes Deployment:** Define the application image, replicas, container port, and deployment configuration in deployment.yaml.
- 7. Create Kubernetes Service:** Define a NodePort service in service.yaml to expose the Flask application outside the cluster.
- 8. Apply the Manifests:** Use kubectl apply to create the Deployment and Service resources.
- 9. Access the API:** Use the assigned NodePort and cluster address to open the Flask API in a browser or API client.

Sample app.py

```
from flask import Flask, jsonify
app = Flask(__name__)

@app.route('/')
def home():
    return jsonify(message='Flask API is running')

@app.route('/health')
def health():
    return jsonify(status='healthy')
```

```
app.run(host='0.0.0.0', port=5000)
```

Sample Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Docker Commands

```
docker build -t <dockerhub-username>/flask-api:latest .
docker login
docker push <dockerhub-username>/flask-api:latest
```

Kubernetes Deployment YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-api
spec:
  replicas: 2
  selector:
    matchLabels:
      app: flask-api
  template:
    metadata:
      labels:
        app: flask-api
    spec:
      containers:
        - name: flask-api
          image: <dockerhub-username>/flask-api:latest
          ports:
            - containerPort: 5000
```

Kubernetes Service YAML

```
apiVersion: v1
kind: Service
metadata:
  name: flask-api-service
spec:
  type: NodePort
  selector:
    app: flask-api
  ports:
    - port: 5000
      targetPort: 5000
      nodePort: 30007
```

Kubernetes Commands

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get deployments
kubectl get pods
kubectl get services
```

Result

The Flask API was containerized using Docker, the image was prepared for Docker Hub, and the application was deployed to a Kubernetes cluster using a Deployment and a NodePort Service. The screenshot shows an

active Kubernetes cluster with the Flask deployment available at 2/2 pods and both pods in the Running state.

SCREENSHOT OF COMPLETED KUBERNETES DEPLOYMENT

Kubernetes [Give feedback](#)

Namespace

default

▼

Cluster

Stop

Edit cluster

Cluster	Cluster type	Nodes	Kubernetes version
Active	kind	3	v1.36.1

Deployments

Name	Status	Pods
flask-ex17	Available	2/2

Pods

Name	Status
flask-ex17-757d8ddcc8-h2xpv	Running
flask-ex17-757d8ddcc8-rpchz	Running

Nodes

Services

Conclusion: The complete workflow from Flask API creation to Docker containerization and Kubernetes deployment was demonstrated successfully.